Data Structures and Algorithms LAB – Spring 2026

(BS-IT-F24 Morning & Afternoon)

Lab # 7 (Online Lab)

Submission Deadline: ***Friday, March 13, 2026. (till 10:00 PM)***

Instructions:

- You must complete all tasks individually. Absolutely NO collaboration is allowed. Any traces of plagiarism/cheating or usage of AI tools will result in an **"F"** grade in this course and lab.
- Attempt the following tasks exactly **in the given order**.
- **Indent** your code properly.
- Use meaningful variable and function names. Use the **camelCase** notation.
- Use meaningful prompt lines/labels for all input/output.
- Make sure that there are **NO** ***dangling pointers*** or ***memory leaks*** in your programs.
- Late submissions will NOT be accepted, whatever your excuse may be.

Submission Procedure:

i. There are 12 tasks in this lab. You are required to submit C++ programs for the **first 8 tasks**. The remaining tasks are for your practice and you are not required to submit them. But I will assume in quizzes and exams that you have solved all of these questions/tasks ☺

ii. For each task, you must submit its code in a single **.CPP** file. So, there should be 8 .CPP files in your submission. The names of these files must be: **Task-1.cpp**, **Task-2.cpp**, …**Task-8.cpp**. Do NOT include any other files in your submission, otherwise your submission will NOT be graded. Don't forget to mention your Name, Roll Number and Section in comments at the top of each CPP file.

iii. Put all the .CPP files created in previous step into a folder. Now, compress this folder in **.ZIP** format. Then, rename the ZIP file ***exactly*** according to the following format:

`Mor-Lab7-BITF24M123` (for Morning section) OR

`Aft-Lab7-BITF24A456` (for Afternoon section),

where the text shown in **BLUE** should be your **complete Roll Number**.

iv. Carefully check your**.ZIP** file to ensure that all above instructions have been followed.

v. Finally, submit the *single* **.ZIP** file through **Google Classroom**. Make sure that you press the **SUBMIT** button after uploading your file.

Note: *If you do not follow the above submission procedure, your Lab will NOT be graded and you will be awarded ZERO marks.*

Carefully note that:

- *All of the following tasks require you to write **recursive** functions. In all of these functions, you are NOT allowed to use any loop.*
- *Moreover, for each recursive function that you implement, you are ALSO required to write a **main** function (driver program) to test its working on several (at least 3) different inputs.*
- *Furthermore, in these recursive functions, you are NOT allowed to declare/use any **global** or **static** variable(s).*

Task # 1

Implement a **recursive** C++ function which takes a positive integer **`val`** as argument and prints that integer on the screen vertically. The prototype of your function should be:

```
void printVertical (int val)
```

For example, the function call **`printVertical(357)`** should display:

```
3
5
7
```

Task # 2

2.1 Implement a **recursive** C++ function which takes an integer array (**`arr`**) and the starting (**`start`**) and ending (**`end`**) indices of that array, and prints all the values present in that array in a zig zag way i.e. initially the first and the last elements will be displayed, then the second and the second-last elements will be displayed, and so on. For example:

If the array contains **`{3,6,2,1,4}`** then the output should be **`3 4 6 1 2`**

If the array contains **`{7,1,9,3,5,2}`** then the output should be **`7 2 1 5 9 3`**

The prototype of your function must be:

```
void printZigZag (int* arr, int start, int end)
```

2.2 Implement a modified version of printZigZag function which takes as arguments the **pointers** to the first and the last element of an integer array. The prototype of this function should be:

```
void printZigZagUpdated (int* pStart, int* pEnd)
```

Task # 3

Implement a **recursive** C++ function which takes an array (**`arr`**), the starting (**`start`**) and ending (**`end`**) indices of that array, and a value (**`key`**) to be searched in that array. This function should determine and return the count of all occurrences of **`key`** in the given array **`arr`**. The prototype of your function should be:

```
int linearSearchUpdated (int* arr, int start, int end, int key)
```

Task # 4

4.1 Implement a **recursive** C++ function which takes a character (**ch**) and a positive integer (**n**) and prints the character **ch**, **n** times on the screen. The prototype of your function should be:

```
void printChar (char ch, int n)
```

For example, calling **printChar('Z',5)** should display **ZZZZZ** on screen.

***Note*:** You are NOT allowed to use any loop in your function.

4.2 Now, implement a **recursive** C++ function which takes a character (**ch**) and a positive integer (**n**) and, and prints a pattern on screen (see below). For example, it should display the following pattern when the arguments are **'*'** and **4**, respectively:

```
****
***
**
*
```

The prototype of your function should be:

```
void printPattern1 (char ch, int n)
```

***Hint*:** Use the function **printChar** that you implemented above in **Task # 4.1**.

4.3 Implement another **recursive** C++ function which prints the following pattern on screen when its arguments are **'#'** and **6**, respectively:

```
#
##
###
####
#####
######
```

The prototype of your function should be:

```
void printPattern2 (char ch, int n)
```

***Hint*:** Use the function **printChar** that you implemented above in **Task # 4.1**.

Task # 5

Implement a **recursive** C++ function which takes an array of integers (**arr**) and the starting (**start**) and ending (**end**) indices of a *portion* (part) of this array, and returns the **index of the largest element** present in that portion of array *arr*. The prototype of your function should be:

```
int findLargestIndex (int* arr, int start, int end)
```

For example, the function call **findLargestIndex(arr,3,8)** should determine and return the ***index*** of the largest element present in the array **arr** between the indices **3** and **8** (both inclusive).

Task # 6

Implement a **recursive** C++ function which takes an integer (**num**) as its only parameter. This function should determine and return the sum of the digits of that integer (**num**). The prototype of your function should be:

```
int sumDigits (int num)
```

For example,

the function call	**sumDigits(357)**	should return	**15**
the function call	**sumDigits(-123)**	should return	**6**
the function call	**sumDigits(0)**	should return	**0**

Task # 7

Implement a **recursive** C++ function which takes two integer arrays and their sizes as arguments, and determines whether these two arrays are equal or not. Note that two arrays are equal if they contain the same number of elements and the elements of both arrays occur in the same order. The prototype of your function should be:

```
bool areArraysEqual (int* a, int aSize, int* b, int bSize)
```

Task # 8

Implement a **recursive** C++ function which takes a c-string and its length as parameters and returns the number of vowels present in that c-string. The prototype of your function should be:

```
int countVowels (char* str, int length)
```

Task # 9

Implement a **recursive** C++ function which takes two integers **a** and **b** as arguments and returns their **product**. The prototype of your function should be:

```
int product (int a, int b)
```

Note: You are NOT allowed to use multiplication operator (*****) in your implementation.

Hint: Think about repeated addition.

Task # 10

Implement a **recursive** C++ function which takes two integers **num** and **den** as arguments and returns the **integer quotient** that will result when **num** is divided by **den**. The prototype of your function should be:

```
int quotient (int num, int den)
```

Hint: Think about repeated subtraction.

Task # 11

Implement a **recursive** C++ function which takes two integers **num** and **den** as arguments and returns the **integer remainder** that will result when **num** is divided by **den**. The prototype of your function should be:

```
int remainder (int num, int den)
```

Task # 12

Implement a **recursive** C++ function which takes a positive decimal (Base 10) number (**num**) as its only parameter. This function should display the binary (Base 2) equivalent of that integer on the screen. The prototype of your function should be:

```
void decimalToBinary (int num)
```

For example,

the function call	**decimalToBinary(6)**	should display	**110**
the function call	**decimalToBinary(84)**	should display	**1010100**
the function call	**decimalToBinary(10)**	should display	**1010**
the function call	**decimalToBinary(25)**	should display	**11001**

VERY IMPORTANT

In the next Lab, you may need some or all of the functions from Today's Lab. So, make sure that you have the working implementation of **ALL** the functions of Today's Lab, when you come to the next Lab.